

Project Initialization and Planning Phase

Date	15 July 2026
Team ID	XXXX
Project Title	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) report

The **Emotion Detection and Learning Support Engine** is an Artificial Intelligence-based system developed to recognize a student's emotional state from free-text descriptions of their learning challenges and provide personalized, empathetic support. The project uses **two independently trained models** - a **BiLSTM** (TensorFlow/Keras) and a fine-tuned **BERT** (PyTorch + HuggingFace Transformers) - to classify emotions across five classes: **Bored, Confident, Confused, Curious, and Frustrated**, including detecting when more than one emotion is present at once (e.g. "Bored + Frustrated"). A keyword-based boosting layer sharpens ambiguous predictions from the BiLSTM model. Based on the detected emotion, the system generates a personalized, field-specific response using the **Google Gemini API** (with an automatic template-based fallback if the API is unavailable), delivered through an interactive **Streamlit** web application. This dual-model approach enhances student engagement and emotional well-being by delivering support that actually matches how the student is feeling, rather than a generic response.

Project Overview	
Objective	The primary objective of the project is to develop a dual-model AI system (BiLSTM and BERT) that accurately identifies a student's emotional state from free-text input across five classes - Bored, Confident, Confused, Curious, and Frustrated - and generates personalized, emotion-aware learning support to improve engagement, motivation, and academic performance.
Scope	The project focuses on emotion recognition using two independently trained deep learning architectures - a Keras-based BiLSTM and a PyTorch-based fine-tuned BERT - combined with a keyword-based boosting layer, and integrated with a Streamlit web application. The system analyzes free-text input, predicts the student's emotional state (including mixed emotions), and provides customized, Gemini-generated learning support in real time, with every interaction logged for future analysis.

Problem Statement	
Description	Students often experience boredom, confusion, frustration, or a lack of confidence during independent study, but existing online learning platforms mainly focus on delivering educational content and rarely consider the emotional state behind a learner's own words, resulting in generic responses and reduced engagement.

Impact	Addressing this gap improves students' emotional well-being, increases learning motivation, provides personalized study support matched to how the student is actually feeling, and gives educators a faster way to identify students who may need help.
---------------	--

Proposed Solution	
Approach	Develop a dual-model AI system using a BiLSTM (Keras) and a fine-tuned BERT (PyTorch/HuggingFace Transformers) to analyze student text, detect emotions (including mixed emotions), and generate personalized, Gemini-powered learning support through a Streamlit web application.
Key Features	• Dual-model AI-based emotion detection using BiLSTM and BERT, run side-by-side for comparison. • Keyword-based boosting layer to sharpen ambiguous predictions. • Mixed-emotion detection (e.g. "Bored + Frustrated") rather than a single forced label. • Personalized, field-specific learning responses generated via the Google Gemini API, with a template-based fallback. • Interactive Streamlit web interface with quick-example prompts. • Session analytics dashboard (Plotly) showing emotional trends over time and by study field. • Continuous interaction logging to CSV for future analysis and retraining.

Resource Requirements		
Resource Type	Description	Specification/Allocation
Hardware		
Computing Resources	CPU/GPU specifications	Intel Core i5/i7 processor for local development and inference; Kaggle Notebooks with GPU acceleration used to train both the BiLSTM and the fine-tuned BERT model
Memory	RAM specifications	Minimum 8 GB RAM
Storage	Space for datasets, trained models, and project files	A few GB free - BERT weights (~255 MB), BiLSTM weights (~18 MB), plus datasets and CSV logs
Software		
Frameworks	Python frameworks	Streamlit (web application); TensorFlow/Keras (BiLSTM model); PyTorch (BERT model)
Libraries	Python libraries	HuggingFace Transformers, NLTK, Pandas, NumPy, scikit-learn, Plotly Express, python-dotenv, google-genai
Development Environment		
Data		

Dataset	Source, size, format	A combined dataset built from GoEmotions, ISEAR, EmoContext, and Empathetic Dialogues, plus template-generated and synthetic examples for underrepresented classes (Bored, Confused, Curious), mapped to the project's five target emotion classes: Bored, Confident, Confused, Curious, Frustrated
---------	----------------------	---

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule.

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	UI & Input	USN-1	As a student, I can select my field of study and enter my learning problem in a text area.	3	High	Tejaswini
Sprint-1	Text Preprocessing	USN-2	As a developer, the system cleans and tokenizes input text (regex + NLTK) before it reaches either model.	2	High	Tejaswini
Sprint-1	BiLSTM Emotion Detection	USN-3	As a student, I want a BiLSTM model to classify my emotional state from my text.	5	High	Tejaswini
Sprint-1	BERT Emotion Detection	USN-4	As a student, I want a fine-tuned BERT model to independently classify my emotional state.	5	High	Tejaswini
Sprint-1	Keyword Boosting	USN-5	As a developer, I want a keyword-based boosting layer to sharpen ambiguous BiLSTM predictions.	3	Medium	Tejaswini
Sprint-1	Quick Examples	USN-6	As a student, I can tap a quick-example prompt instead of typing my own problem.	2	Low	Tejaswini
Sprint-2	AI Response Generation	USN-7	As a student, I want a personalized response generated by the Gemini API based on my detected emotion.	5	High	Tejaswini
Sprint-2	Fallback Handling	USN-8	As a student, I still receive a helpful template-based response if the Gemini API call fails.	3	Medium	Tejaswini
Sprint-2	Mixed Emotion Detection	USN-9	As a student, I want to see a secondary emotion (e.g. "Bored + Frustrated") when it crosses a confidence threshold.	5	Medium	Tejaswini
Sprint-2	Response Strategy	USN-10	As a student, I receive a suggested next-step strategy matched to my detected emotion.	3	Medium	Tejaswini
Sprint-2	Model Comparison View	USN-11	As an educator/TA, I can expand "Analysis Details" to compare BiLSTM vs BERT predictions side-by-side.	4	Medium	Tejaswini

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-3	Analytics Dashboard	USN-12	As a student, I want a pie chart of my emotion distribution across the session.	5	Medium	Tejaswini
Sprint-3	Analytics Dashboard	USN-13	As a student, I want a line chart of my emotional journey over time.	4	Medium	Tejaswini
Sprint-3	Analytics Dashboard	USN-14	As an educator, I want a bar chart of emotions broken down by study field and by model.	3	Low	Tejaswini
Sprint-3	Interaction Logging	USN-15	As an administrator, every interaction is appended to CSV files for future analysis or retraining.	5	Medium	Tejaswini
Sprint-3	Session History	USN-16	As a student, I can view recent sessions in the sidebar and clear my history.	3	Low	Tejaswini
Sprint-4	Bug Fixing	USN-17	As a developer, I fixed a missing softmax normalization bug in BiLSTMModel.predict_proba().	3	High	Tejaswini
Sprint-4	Bug Fixing	USN-18	As a developer, I fixed a keyword collision bug where "understand" wrongly matched both Confident and Confused.	3	High	Tejaswini
Sprint-4	Bug Fixing	USN-19	As a developer, I fixed an unconditional class-weight bias that skewed BERT toward "Confident" regardless of input.	4	High	Tejaswini
Sprint-4	Code Cleanup	USN-20	As a developer, I removed debug output and reorganized function scope for the production build.	2	Medium	Tejaswini
Sprint-4	Documentation	USN-21	As a developer, I wrote README.md and PROJECT_ANALYSIS_REPORT.md documenting setup, architecture, and known limitations.	4	Medium	Tejaswini
Sprint-4	Deployment Prep	USN-22	As a developer, I prepared the app for cloud deployment (candidate: Streamlit Community Cloud / Hugging Face Spaces).	4	Low	Tejaswini

Project Tracker, Velocity & Burndown Chart (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	20	5 Days	01 Jul 2026	05 Jul 2026	20	05 Jul 2026
Sprint-2	20	5 Days	06 Jul 2026	10 Jul 2026	20	10 Jul 2026
Sprint-3	20	5 Days	11 Jul 2026	15 Jul 2026	20	15 Jul 2026

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-4	20	5 Days	16 Jul 2026	20 Jul 2026	20	20 Jul 2026

Velocity:

Imagine we have a 10-day sprint duration, and the velocity of the team is 20 (points per sprint). Let's calculate the team's average velocity (AV) per iteration unit (story points per day): $AV = \text{velocity} / \text{sprint duration} = 20 / 10 = 2$ story points per day.

For this project, each sprint ran over a 5-day duration with a velocity of 20 story points per sprint, giving an average velocity of $AV = 20 / 5 = 4$ story points per day. All 4 sprints ran back-to-back from 01 Jul 2026 to 20 Jul 2026, matching the project's fixed deadline of 20 July 2026, with no carried-over story points.

Burndown Chart:

A burndown chart is a graphical representation of work left to do versus time. It is often used in agile software development methodologies such as Scrum. However, burndown charts can be applied to any project containing measurable progress over time. Since all 4 sprints in this project completed their full 20 planned story points by each sprint's end date, the burndown for every sprint traces a steady line from 20 remaining points down to 0 by the planned end date, with no schedule slippage across the project.